

Smart Tests with Python and pytest

20 minute read

Author(s)	Details
Xhesi Galanxhi (xgalanxhi)	Team: PS Date: 2026-03-24
Required Check(s)	Status
PS Official	Pending
ENG Approval	Pending

Important

CRITICAL REQUIREMENT (INTERNAL FACING)

PTSV2 (Predictive Test Selection v2) must be enabled for your CloudBees Unify (CloudBees.io) organization in order to follow this guide.

To enable PTSv2 for your org/suborg:

Send a Slack message to the **#team-smart-tests-se** channel requesting activation. Be sure to include your org/suborg ID.

Finding your org/suborg ID:

- Log in to [cloudbees.io](#)
- Navigate to **Admin Settings > Organization Profile**
- Copy your **Organization ID**

Important note about sub-organizations:

If you have multiple sub-organizations, PTSv2 must be enabled separately for each suborg where you intend to use Smart Tests Predictive Test Selection. Enabling PTSv2 on a parent organization does not automatically enable it for child sub-organizations—each suborg must be explicitly activated.

Qualification Criteria

Overall Requirements

Before proceeding with Smart Tests adoption, the customer must meet ALL of the following requirements:

1. Long-Running Test Suite

Requirement	Details
Minimum Runtime	Full test suite must take 15+ minutes to execute
Rationale	Smart Tests provides the most value when test execution time is significant. Shorter test suites (<15 minutes) may not justify the implementation effort.
How to Measure	Review recent CI/CD pipeline execution times for test stages

Submit Feedback

Tip

Ideal Candidates

- Test suites taking 30-60+ minutes: High value opportunity
- Test suites taking 15-30 minutes: Good candidate
- Test suites taking <15 minutes: Consider future growth or wait until suite grows

2. Independent Tests

Requirement	Details
Test Independence	Tests must be independently executable without dependencies on test execution order
Rationale	Smart Tests creates subsets of tests to run. If tests depend on other tests running first (shared state, execution order), subsetting will cause failures.
How to Validate	Tests can be run in isolation or random order without affecting results

Warning

Common Anti-Patterns That Disqualify

- Tests that rely on shared global state modified by other tests
- Tests that must run in specific order (Test B depends on Test A's side effects)
- Test suites using `pytest.mark.dependency` or similar ordering mechanisms
- Setup/teardown logic that assumes all tests run together

If these patterns exist, customer must refactor tests before Smart Tests adoption.

3. Git Version Control

Requirement	Details
Version Control	Customer must be using Git for source control
Rationale	Smart Tests analyzes code changes via <code>git diff</code> to predict affected tests. Git is the only supported VCS.
Supported	GitHub, GitLab, Bitbucket, Azure Repos, self-hosted Git servers
Not Supported	Subversion (SVN), Mercurial, Perforce, TFS (non-Git)

4. Test File Organization

Requirement	Details
File Structure	Tests must be organized across multiple test files , not a single monolithic file
Rationale	Smart Tests maps code changes to test files. A single file containing all tests prevents effective subsetting.
Good Structure	Tests organized by feature, module, layer, or functional area across multiple files
Disqualifying Structure	Single <code>test_all.py</code> or <code>tests.js</code> file containing hundreds/thousands of tests

Acceptable Organization Patterns

✓ By feature:
tests/auth/test_login.py
tests/auth/test_permissions.py
tests/billing/test_invoices.py

✓ By layer:
tests/unit/test_services.py
tests/integration/test_api.py
tests/e2e/test_workflows.py

✓ By module:
tests/test_user_service.py
tests/test_project_service.py
tests/test_issue_service.py

▼

Supported Test Frameworks

The customer must be using one of the officially supported test frameworks listed below. Smart Tests CLI integration varies by framework.

Framework Compatibility Matrix

Test Framework	test_runner Value	Additional Setup Required	Language/Ecosystem
Android CTS	N/A	Zero Input Only	Android
Android Debug Bridge (ADB)	adb	No	Android
Ant	ant	Yes	Java
Bazel	bazel	No	Multi-language
Behave	behave	No	Python (BDD)
CTest	ctest	No	C/C++ (CMake)
Cucumber	cucumber	No	Multi-language (BDD)
Cypress	cypress	No	JavaScript (E2E)
dotnet test	N/A	Zero Input Only	.NET/C#
Flutter	flutter	No	Dart
GoogleTest	googletest	No	C++
Go Test	go-test	No	Go
Gradle	gradle	No	Java/Kotlin
Gradle + TestNG	gradle	Yes	Java/Kotlin
Jest	jest	No	JavaScript/TypeScript
Karma	karma	No	JavaScript/TypeScript
Maven	maven	No	Java
Maven + TestNG	maven	Yes	Java

Test Framework	test_runner Value	Additional Setup Required	Language/Ecosystem
Minitest	minitest	No	Ruby
NUnit Console Runner	nunit	No	.NET/C#
Playwright	playwright	No	JavaScript/TypeScript
prove (Perl)	prove	No	Perl
pytest	pytest	No	Python
Robot Framework	robot	No	Python (Keyword-driven)
RSpec	rspec	No	Ruby
Vitest	vitest	No	JavaScript/TypeScript
XCTest	N/A	Zero Input Only	Swift/Objective-C (iOS)

Note

Framework-Specific Considerations

"Additional Setup Required: Yes" indicates that extra dependencies or configuration files are needed beyond command-line arguments:

- **Gradle + TestNG:** Requires `launchable-testng` dependency in `build.gradle`
- **Maven + TestNG:** Requires `launchable-testng` dependency in `pom.xml`
- **Ant:** Requires modifications to `build.xml` file

"Additional Setup Required: No" means integration uses only command-line arguments and piped commands.

"Zero Input Only" frameworks (Android CTS, dotnet test, XCTest) use an alternative subsetting method that doesn't require test lists. See CloudBees documentation for Zero Input Subsetting.

CI/CD Platform Requirements

Platform Compatibility

Smart Tests CLI is **CI/CD platform agnostic**. Customer can use any CI/CD tool that meets the following requirements:

Requirement	Details
Python 3.7+	Smart Tests CLI requires Python 3.7 or later installed on CI runners
Java Runtime	Java runtime environment required for Smart Tests CLI execution
CLI Execution	CI platform must allow running shell commands (<code>smart-tests</code> command)
Environment Variables	CI platform must support setting secrets/environment variables (for Smart Tests API token)

PS Tested CI/CD Platforms

- CloudBees Unify
- GitHub Actions
- CloudBees CI
- Jenkins

Validation Steps

To verify CI platform compatibility:

1. Check Python version: `python --version` (must be 3.7+)
2. Check Java availability: `java -version`
3. Test CLI installation: `pip install smart-tests-cli && smart-tests --version`
4. Verify network access to CloudBees API endpoints

If all commands succeed, the platform is compatible.

Overview

This guide teaches CloudBees Smart Tests predictive test selection concepts using pytest-based applications as the implementation pattern. While we use a demonstration application for concrete examples, the concepts and integration patterns apply to any Python application using pytest for testing.

Demonstration Repository: `\\https://github.com/cloudbees-ps/issues-tracker-app`

What This Journey Provides

This journey shows an **end-to-end implementation** of Smart Tests with pytest, including:

- **Working CI Integration:** Complete GitHub Actions workflow with Smart Tests CLI
- **Example Codebase:** Python application using pytest (Smart Tests-supported framework)
- **Demo Patches:** Pre-built code changes that simulate real development work

Why Patches? To demonstrate Smart Tests' value, we need code changes that break tests. Normally, this happens naturally as developers commit code throughout the day. Instead of requiring you to manually modify code to break tests, this journey provides **patches** — pre-packaged commits with realistic code changes that you can apply with a single click. Each patch adds commits to your branch simulating what a developer would actually do (e.g., adding validation rules, modifying business logic). Smart Tests then analyzes these commits to predict which tests will fail.

All materials are included in the accompanying repository.

By the end of this guide, you will:

- Understand core Smart Tests concepts: observation mode, prediction accuracy, and test selection
- Learn pytest-specific integration patterns with Smart Tests CLI
- See how code changes map to test predictions in real scenarios
- Know how to implement Smart Tests in your own pytest-based applications
- Interpret Smart Tests metrics to validate effectiveness
- Apply patterns from the demo to your organization’s test suites

Target Audience: Teams using Python and pytest for testing

Estimated Time: 20-30 minutes (including hands-on exploration)

Prerequisites

General Requirements (Any Customer)

To Use Smart Tests with pytest:

Requirement	Notes
Python 3.7+	Smart Tests CLI requires modern Python
pytest installed	Any version, with any plugins
CI/CD pipeline	GitHub Actions, Jenkins, GitLab CI, etc.
Git repository	For change detection
CloudBees account	Edition requirement has not been confirmed. Contact your CloudBees account team for licensing details.
PTSV2 enabled	Contact CloudBees to enable (see below)

Demo-Specific Requirements

For the Hands-On Demo Only:

Requirement	Notes
GitHub account	To fork the demo repository
GitHub Actions enabled	Ability to run workflows in your fork
Time commitment	15-20 minutes for core demo

Parameterized Values

The following values appear throughout this guide. Replace them with your actual values before running any commands.

Placeholder	Description	Where to Find
<YOUR_SMART_TESTS_TOKEN>	CloudBees API token for Smart Tests CLI authentication	CloudBees UI: Smart Tests > Settings > Create a Workspace API Key

Implementation

Initial Setup

Fork the Repository

Steps:

- Navigate to: \\https://github.com/cloudbees-ps/issues-tracker-app
- Click **Fork** button (top right)
- Select your GitHub account
- Wait for fork to complete

Result: Personal copy of repository under your GitHub account

Enable GitHub Actions

Steps:

- Go to your forked repository

2. Click **Actions** tab
3. If prompted, click "**I understand my workflows, go ahead and enable them**"

Workflows Available:

- **Tests:** Runs unit, integration, and E2E tests with Smart Tests
- **Apply CI Demo Patch:** Applies breaking changes to demonstrate CI behavior

Configure Smart Tests Token

Obtain CloudBees API Token

1. Log in to <https://cloudbees.io>
2. Navigate to **Smart Tests > Settings**
3. Click **Create a Workspace API Key**
4. Enter key name: `demo-token`
5. Copy the generated token

Note

Store this token securely. You'll only see it once.

Add Token to GitHub

1. Go to your forked repository
2. Navigate to **Settings > Secrets and variables > Actions**
3. Click **New repository secret**
4. Configure:
 - **Name:** `SMART_TESTS_TOKEN`
 - **Value:** Paste your CloudBees API token
5. Click **Add secret**

Verification: Secret appears in repository secrets list

Hands-On: Observing Smart Tests

Create Your Working Branch

1. Go to **Code** tab in your forked repository
2. Click branch dropdown (currently shows "main")
3. Type: `patch-demo` in the text field
4. Click **Create branch: patch-demo from main**

Note

Branch names must start with `patch-` for safety (workflow restriction). You can also use `patch-demo-1` , `patch-test` , or any name starting with `patch-` .

This creates your working branch where you'll apply changes. Smart Tests will analyze commits on this branch to predict which tests are affected.

Establish Baseline on Your Branch

1. Navigate to **Actions** tab
2. Click **Tests** workflow

3. Click **Run workflow** button
4. Configure:
 - **Branch:** `patch-demo`
 - **mode:** `observation` (default - keep as-is)
 - **optimization_target:** `--target 50%` (default - keep as-is)
5. Click **Run workflow**

Wait for the workflow to complete (~2-3 minutes).

Note

Keep the default workflow parameters for now. You'll experiment with different options later after understanding the baseline behavior.

Note

What just happened:

This established your baseline — a "clean" test run on the `patch-demo` branch with all tests passing. Smart Tests recorded:

- All 140+ tests executed (unit + integration + E2E)
- Execution times for each test
- Test results uploaded to CloudBees

Since there are no commits with code changes yet on this branch, there are no predictions. This baseline establishes the starting point.

View Initial Test Session

1. Open <https://cloudbees.io>
2. Navigate to **Predictive Test Selection > Observation Mode**
3. Find the latest session for the `patch-demo` branch

You should see:

- 140+ tests passed
- 3 test suites (unit, integration, e2e)
- ~2-3 minutes duration
- No predictions (no code changes yet on this branch)

Tip

This dashboard shows all test execution data. Each CI build creates a new session record. Once you make code changes on this branch, Smart Tests will analyze the commits and generate predictions.

Apply Code Changes

1. Navigate to **Actions** tab
2. Click **Apply CI Demo Patch** workflow
3. Click **Run workflow**
4. Configure:
 - **Branch:** `patch-demo`
 - **Action:** `apply`
 - **Patch:** `01-require-project-description.patch`
5. Click **Run workflow**

Wait for the workflow to complete (~30 seconds).

Note

What was changed:

This patch adds a new commit to your `patch-demo` branch that simulates a developer adding validation requirements:

- `app/models/project.py` - database model changes
- `app/schemas/project.py` - validation schema updates
- `app/services/project_service.py` - business logic modifications

Expected impact: * Tests creating projects without descriptions: FAIL * Tests with short descriptions: FAIL * Unrelated tests: PASS

Smart Tests will analyze this commit (the diff from the previous state of the branch) to predict which tests are likely to fail.

Run Tests with Changes

1. Navigate to **Actions** tab
2. Click **Tests** workflow
3. Click **Run workflow**
4. Configure:
 - Branch:** `patch-demo`
 - mode:** `observation` (keep default)
 - optimization_target:** `--target 50%` (keep default)
5. Click **Run workflow**

Wait for the workflow to complete (~2-3 minutes).

Expected result: Some tests will FAIL (8-10 failures). This is correct — the code changes intentionally broke tests.

Note

What Smart Tests is doing behind the scenes:

1. **Collect tests:** `pytest --collect-only` lists all 140+ tests

2. **Analyze commits:** Examines the recent commits on `patch-demo` branch to see what code changed

3. **Generate predictions:** AI model predicts ~70 tests (~50%) are likely affected by these code changes

4. **Run ALL tests:** In observation mode, all 140+ tests execute (not just predicted)

5. **Record everything:** Uploads results + predictions to CloudBees

Key insight: Smart Tests analyzes the commit history on the branch to understand what changed, then predicts which tests are likely to fail. Even though tests failed, Smart Tests recorded which tests were predicted, which actually ran (all of them), and which failed. You can now compare: did the predicted subset include all failures?

Warning

Understanding Observation Mode

In observation mode: * **All tests run** - no tests are skipped * Predictions are created but not acted upon * You validate accuracy with zero risk * Time savings are projected, not realized

This lets you prove Smart Tests works before relying on it.

Analyze Prediction Results

1. Open <https://cloudbees.io>
2. Navigate to **Predictive Test Selection > Observation Mode**

3. Find the latest session for the `patch-demo` branch

Review the dashboard showing:

Actual Execution: * Tests Run: 140 * Tests Failed: 8-10 * Duration: ~3 minutes

Smart Tests Prediction: * Tests Predicted: ~70 (50% of suite) * Failures Captured: 8-10 (should be all failures) * Predicted Duration: ~1.5 minutes * Projected Time Savings: ~1.5 minutes (50%)

Tip

Key metrics to understand:

- **Prediction Accuracy:** Did the predicted subset include all failures? Look for >90% accuracy.
- **Test Selection Rate:** What % of tests were predicted? (~50% means significant savings potential)
- **Time Savings:** How much faster would the predicted subset have been?

What you should see: * 8-10 tests failed (project validation logic) * Predicted subset includes all failures (100% accuracy) * ~50% test selection rate * Significant time savings projection

This demonstrates Smart Tests correctly identified affected tests while running only half the suite would still catch all failures.

Experiment with Different Scenarios

Now that you’ve seen how Smart Tests works with one patch, try experimenting with different patches to see how Smart Tests adapts to various types of code changes.

Available Patches:

Patch	Description	Expected Test Impact
<code>01-require-project-description.patch</code>	Require 50+ char descriptions	8 failures
<code>02-require-long-issue-titles.patch</code>	Require 20+ char issue titles	~10 failures
<code>03-require-long-comments.patch</code>	Require 15+ char comments	~10 failures
<code>04-require-corporate-email.patch</code>	Require corporate email domains	~7 failures

How to experiment:

Apply a different patch: . Go to **Actions > Apply CI Demo Patch** . Choose a different patch (e.g., `02-require-long-issue-titles.patch`) . Run the **Tests** workflow again (keep default mode: `observation` and target: `--target 50%`) . View results in CloudBees dashboard

Revert to clean state: . Go to **Actions > Apply CI Demo Patch** . Set **action** to `revert` . Specify the patch you want to remove . Run **Tests** workflow to verify all tests pass again

Note

Reverting adds a new commit that undoes the changes. Smart Tests will analyze this revert commit too, so you may still see predictions based on the code being changed back.

Try multiple patches: . Apply one patch → run tests → view predictions . Apply another patch on top → run tests → see how predictions change . Revert patches one by one to return to baseline

Tip

What to observe:

- Each patch represents a different type of code change (validation rules, business logic, etc.)
- Notice how Smart Tests adjusts its predictions based on what code was modified
- Some patches affect more tests than others (e.g., 8 vs. 10 failures)
- Combining multiple patches (~35 total failures) shows Smart Tests handling complex, multi-domain changes
- Reverting patches adds new commits that Smart Tests also analyzes

Experiment with Workflow Options

Now that you understand how Smart Tests works, experiment with the workflow configuration options to see how they affect behavior.

The **Tests** workflow accepts two input parameters:

Parameter	Options	What It Controls
mode	<code>observation</code> (default) <code>production</code>	Whether all tests run or only predicted subset
optimization_target	<code>--target 50%</code> (default) <code>--target 30%</code> <code>--target 70%</code> <code>--confidence 90%</code> <code>--time 10m</code>	How Smart Tests selects test subset

Understanding Optimization Targets:

Smart Tests offers three types of optimization targets, each suited for different scenarios:

- Percentage time target** (`--target %`): * Returns a percentage of the expected test duration * Best for test suites with variable duration * Example: `--target 20%` runs 20% of expected duration (20 min if full suite = 100 min) * **Used in this demo** - most flexible option
- Confidence target** (`--confidence %`): * Targets a probability of catching failing sessions * Best for test suites with consistent test lists * Example: `--confidence 90%` runs tests up to the duration that gives 90% confidence * Based on the confidence curve in CloudBees dashboard
- Fixed time target** (`--time`): * Sets a maximum test runtime * Best for test suites with stable run duration * Example: `--time 10m` runs up to 10 minutes of most relevant tests * Useful when you have a hard time constraint

NOTE

Choosing the right optimization target:

- Use `--target %` (percentage) when:
 - Your test sessions vary in duration
 - You want predictable subset size relative to full suite
 - This demo uses percentage targets - most flexible option
- Use `--confidence %` when:
 - Your test list is consistent across runs
 - You want to target a specific probability of catching failures
 - You're comfortable with variable subset duration
- Use `--time` (fixed duration) when:
 - Your test suite has stable total duration
 - You have hard time constraints (e.g., "max 10 minutes")
 - You want predictable absolute runtime

How to experiment:

Try Production Mode (actual test selection):

1. Go to **Actions > Tests**
2. Click **Run workflow**
3. Configure:
 - **Branch:** `patch-demo`
 - **mode:** `production`
 - **optimization_target:** `--target 50%` (keep default)
4. Run workflow

Warning

In production mode, only the predicted subset of tests will run. This is where time savings are realized, but also where risk exists if predictions miss failures.

You'll notice the workflow completes faster (~1.5 minutes instead of 3 minutes) because only ~70 tests run instead of all 140+.

Try Different Optimization Targets:

Experiment with percentage targets: . Run workflow with `--target 30%` (more aggressive - fewer tests, bigger savings) . Run workflow with `--target 70%` (more conservative - more tests, smaller savings) . Compare results in CloudBees dashboard

Try other target types (optional): . Run workflow with `--confidence 90%` (if available - targets 90% probability of catching failures) . Run workflow with `--time 5m` (if available - runs up to 5 minutes of tests) . View how different target types affect test selection

Tin

What to observe:

- **Observation mode** (`mode: observation`):
 - All tests run every time
 - Predictions are created but not acted upon
 - Zero risk - validates accuracy before production use
 - Time savings are projected, not realized
- **Production mode** (`mode: production`):
 - Only predicted tests run
 - Actual time savings achieved
 - Small risk if predictions miss failures
 - Use after validating accuracy in observation mode
- **Percentage target** (`--target %`):
 - `--target 30%` : Aggressive - runs 30% of expected duration (bigger savings, higher risk)
 - `--target 50%` : Balanced - runs 50% of expected duration (default)
 - `--target 70%` : Conservative - runs 70% of expected duration (smaller savings, lower risk)
 - Best for variable-duration test suites
- **Confidence target** (`--confidence %`):
 - Example: `--confidence 90%` = 90% probability of catching failures
 - Duration determined by confidence curve
 - Best for test suites with consistent test lists
 - Can return all tests if total duration < curve's x-axis value
- **Fixed time target** (`--time`):
 - Example: `--time 10m` = max 10 minutes of most relevant tests
 - Useful when you have hard time constraints
 - Best for test suites with stable run duration
 - Can return all tests if total duration < time limit

Hands-On Demo Complete

You've completed:

1. ✓ Created a working branch (`patch-demo`)
2. ✓ Established baseline tests on the branch (all passing)
3. ✓ Applied code changes via patches (new commits added)
4. ✓ Ran tests with Smart Tests analyzing the commits
5. ✓ Viewed predictions showing which tests were likely to fail
6. ✓ Analyzed prediction accuracy and time savings
7. ✓ Experimented with different patches (optional)
8. ✓ Experimented with workflow options: observation vs production mode, optimization targets (optional)

What you should see in CloudBees:

- Multiple test sessions on `patch-demo` branch showing:

- Baseline session: no predictions, all tests passed
 - Sessions after patches: predictions generated based on commits, varying test failures
 - Observation mode sessions: all tests run, predictions shown for validation
 - Production mode sessions: only predicted tests run, actual time savings realized
 - Different optimization targets (`--target 30%`, `50%`, `70%`, etc.): varying subset sizes
- Prediction accuracy >95% across different change types
 - Time savings varying by optimization target:
 - `--target 30%` : ~70% time savings (aggressive)
 - `--target 50%` : ~50% time savings (balanced, default)
 - `--target 70%` : ~30% time savings (conservative)
 - Clear view of which tests were predicted vs. which actually failed for each scenario
 - Faster execution times in production mode sessions (~1.5 min vs 3 min for default 50% target)

Core demo complete! You’ve experienced the essential Smart Tests workflow: Smart Tests analyzes commits on your branch to predict which tests will fail. You’ve seen how it adapts to different types of code changes, and how optimization targets let you balance speed vs. accuracy.

Understanding the CI Integration

Now that you’ve completed the hands-on demo and are familiar with how Smart Tests displays data, observation mode, and subset target selection, let’s take a closer look at what happens behind the scenes to send data to Smart Tests.

The CI Workflow Steps

In the GitHub repository, open `.github/workflows/tests.yml` to see the complete CI integration. The workflow follows these key steps:

1. Record Build (`smart-tests record build`)

```
smart-tests record build --build ${GITHUB_RUN_ID}
```

Groups related test sessions together. The build ID ties multiple test sessions (e.g., unit, integration, e2e) to a single CI run.

2. Record Session (`smart-tests record session`)

```
smart-tests record session \
  --build ${GITHUB_RUN_ID} \
  --observation \
  --test-suite pytest-unit > session.txt
```

Creates a test session in CloudBees. The `--observation` flag tells Smart Tests to run all tests while generating predictions for validation.

3. Collect Tests (`pytest --collect-only`)

```
pytest --collect-only -qq tests/ > all_tests.txt
```

Generates a list of all available tests. This list is passed to Smart Tests for analysis.

4. Generate Subset (`smart-tests subset`)


```
smart-tests subset pytest < all_tests.txt > subset.txt
```

Analyzes commits and predicts which tests are affected. The subset file contains the predicted tests based on code changes and the optimization target.

5. Run Tests (`pytest`)

```
pytest @subset.txt \  
  -o junit_family=legacy \  
  --junit-xml=results.xml
```

Executes tests using pytest. In observation mode, this runs all tests. In production mode, only the tests in `subset.txt` are executed.

6. Record Results (`smart-tests record tests`)

```
smart-tests record tests pytest results.xml
```

Uploads test results to CloudBees. Smart Tests compares actual failures with predictions to calculate accuracy.

pytest-Specific Requirements

Smart Tests expects test results in a specific format. pytest by default does not return results in a Smart Tests-compatible format, so we need to add the `junit_family=legacy` flag:

```
pytest -o junit_family=legacy --junit-xml=results.xml
```

This ensures pytest outputs JUnit XML in the legacy format that Smart Tests can parse correctly.

Note

Framework-specific quirks: Other supported test frameworks may have similar requirements. For example: * Some frameworks require specific output plugins * Others need particular XML schema versions * Configuration files (pytest.ini, pyproject.toml) can set these defaults

Refer to the Smart Tests documentation for framework-specific integration details.

Key Configuration Points

When adapting this workflow to your environment:

- **Build ID:** Use your CI platform’s build identifier (`${BUILD_ID}` , `${CI_PIPELINE_ID}` , etc.)
- **Test Suite Names:** Choose meaningful names (`pytest-unit` , `pytest-integration` , etc.)
- **Test Collection:** Adjust paths and filters for your test organization
- **Output Format:** Ensure your test framework outputs compatible results (JUnit XML legacy format)
- **Observation vs Production:** Start with `--observation` to validate, then remove it for production mode

Deep Dive: Concepts and Patterns

Note

This section provides in-depth conceptual material and implementation patterns.

If you completed the hands-on demo and want to understand the "why" behind Smart Tests, read on. Otherwise, you can skip to [Summary](#) for the big picture or jump to specific topics using the table of contents.

Core Concepts

Predictive Test Selection Fundamentals

The Problem:

As test suites grow, running all tests on every code change becomes increasingly expensive:

- Hundreds or thousands of tests taking 10+ minutes to execute
- Most code changes only affect a small subset of tests
- Developers wait for results even when changes are localized
- CI infrastructure costs scale with test suite size

The Smart Tests Solution:

Instead of running all tests, Smart Tests uses AI to predict which tests are relevant to specific code changes:

1. **Analyze:** Examine git diff to understand what changed
2. **Predict:** Use ML models to identify affected tests
3. **Select:** Run only the predicted subset
4. **Learn:** Validate predictions and improve accuracy

Key Benefits:

Benefit	Customer Impact
50-80% Faster Feedback	Developers get test results in minutes instead of 10-30+ minutes
Cost Reduction	Lower CI compute costs by running fewer tests
Maintained Quality	High accuracy (>90%) ensures regressions are caught
Scalability	Test suite can grow without proportional time increase
Developer Experience	Less waiting, faster iteration cycles

Observation Mode: Risk-Free Validation

Concept: Run all tests while Smart Tests creates predictions in parallel, allowing validation without risk.

How It Works:

What Happens in Your CI	What Smart Tests Does
All tests run normally	Creates predicted subset based on code changes
Test results recorded	Uploads actual results to CloudBees
Build succeeds/fails as usual	Compares predictions vs. actual failures
No behavior change	Shows "what-if" analysis in UI

Why This Matters for Customers:

- **Zero Risk:** Your CI pipeline continues unchanged
- **Proof of Value:** See actual time savings before committing
- **Accuracy Validation:** Verify predictions match your codebase patterns
- **Stakeholder Buy-In:** Show concrete data to management

Metrics You’ll See:

- 1. **Prediction Accuracy:** Did predicted subset include all failures?
- 2. **Time Savings:** How much faster would predicted subset have been?
- 3. **Confidence Score:** Smart Tests' certainty in predictions
- 4. **Coverage Analysis:** Which tests were/weren’t selected

Warning

COMMON MISCONCEPTIONS: Observation Mode

Misconception #1: "Observation mode is just for demo purposes."

Reality: Observation mode is a critical production phase. Most successful deployments run observation mode for 1-2 weeks to validate predictions before enabling test selection.

Misconception #2: "In observation mode, Smart Tests doesn’t do anything."

Reality: Smart Tests actively creates predictions, records data, and validates accuracy. You’re building confidence while maintaining zero risk.

Misconception #3: "If accuracy is high in observation mode, I can skip to 100% adoption."

Reality: Start with non-critical branches or projects first. Gradual rollout (pilot, expand, majority, full) reduces risk and builds team confidence.

Misconception #4: "Observation mode slows down my CI."

Reality: Smart Tests operations (recording metadata, creating predictions) add negligible overhead (<5 seconds). Tests run at normal speed.

Production Mode: Enabling Test Selection

Concept: Once you’re confident in Smart Tests' accuracy, remove the `--observation` flag to run only the predicted subset, achieving actual time savings.

From Observation to Production:

Observation Mode	Production Mode
<code>--observation</code> flag used	<code>--observation</code> flag removed
All tests run	Only predicted subset runs
Time savings are projected	Time savings are realized
Zero risk (all tests execute)	Minimal risk (high accuracy validated)
Used for validation	Used for daily development

How It Works:

```
# Step 1: Record session (remove --observation flag)
smart-tests record session \
  --build $BUILD_ID \
  --test-suite pytest-unit > session.txt

# Step 2: Collect all tests (same as before)
pytest --collect-only -qq tests/ > all_tests.txt

# Step 3: Create predicted subset (same as before)
smart-tests subset pytest < all_tests.txt > subset.txt

# Step 4: Run ONLY the predicted subset
pytest @subset.txt \
  -c junit-family=legacy \
```

Key Difference:

In production mode, pytest executes **only the tests listed in subset.txt** (the predicted subset), not all tests. This is where time savings are achieved.

Example - Demo App Scenario:

Observation Mode (validation phase):
All tests collected: 140 tests
Predicted subset: 70 tests (50%)
Tests executed: ALL 140 tests, 3 minutes
Result: Validated that predicted 70 would catch all failures

Production Mode (after validation):
All tests collected: 140 tests
Predicted subset: 70 tests (50%)
Tests executed: ONLY 70 tests, 1.5 minutes
Result: 50% time savings realized

When to Transition to Production Mode:

Criterion	Recommendation
Accuracy	Consistently >85-95% over 1-2 weeks
Test Sessions	50-100+ sessions in observation mode
False Negatives	Minimal or zero missed failures
Team Confidence	Developers and stakeholders trust predictions
Rollout Strategy	Start with non-critical branches, expand gradually

Tip

Gradual Production Rollout

Best Practice: Don't switch everything to production mode at once.

Recommended Approach:

- Week 1-2:** All branches in observation mode
- Week 3:** Enable production mode for feature branches only
- Week 4:** Enable for pull request validation builds
- Week 5:** Enable for develop/staging branches
- Week 6+:** Enable for main/production branches

This gradual rollout minimizes risk while building team confidence.

09/06/2026, 13:32

Smart Tests with Python and pytest

Warning

Fallback Options

Always maintain the ability to run full test suite:

- Keep full suite workflow available for critical changes
- Add manual trigger to bypass subsetting for release builds
- Monitor accuracy continuously and revert to observation if it degrades
- Document process for disabling predictions in emergencies

Example: Add a workflow input parameter `skip_smart_tests: false` that runs full suite when set to `true` .

pytest Integration Pattern

Why pytest:

pytest is one of the most popular Python testing frameworks, used across industries:

- Mature ecosystem with extensive plugins
- Supports unit, integration, functional, and E2E tests
- Flexible test discovery and execution
- Built-in fixtures and parametrization
- Industry standard for Python applications

Smart Tests + pytest Compatibility:

- Works with standard pytest installations
- No changes to existing test code required
- Supports all pytest plugins and extensions
- Compatible with pytest-xdist (parallel execution)
- Integrates with pytest markers and fixtures

Example Application (Issues Tracker):

We use a real FastAPI application with pytest:

- 140+ pytest tests
- Unit tests (services)
- Integration tests (APIs)
- E2E tests (Playwright)
- FastAPI + SQLAlchemy stack

12

KNOWLEDGE CHECK: Core Concepts

Test your understanding of core concepts:

- What problem does Smart Tests solve?**
 - ☐ Making tests run faster by parallelizing them
 - ☒ Intelligently selecting which tests to run based on code changes
 - ☐ Automatically fixing failing tests
 - ☐ Replacing the need for integration tests
- What is observation mode?**
 - ☐ A mode where Smart Tests only watches but doesn't record data
 - ☒ Running all tests while Smart Tests creates predictions for validation
 - ☐ Running only the predicted subset of tests
 - ☐ A debugging mode for Smart Tests CLI
- Does Smart Tests require changes to your pytest test code?**
 - ☐ Yes, you need to add decorators to each test
 - ☐ Yes, you need to reorganize your test files
 - ☒ No, it uses standard pytest features without code changes
 - ☐ Yes, but only for integration tests
- What does Smart Tests analyze to create predictions?**
 - ☐ Only the test execution times
 - ☐ Only the test file names
 - ☒ The git diff (code changes) between commits
 - ☐ Developer commit messages
- Why is observation mode important?**
 - ☐ It's required by CloudBees licensing
 - ☐ It makes tests run faster
 - ☒ It validates prediction accuracy before relying on test selection
 - ☐ It's only needed for the demo

Answers at bottom of page

Technical Integration Patterns

The Six-Step Integration Pattern

Demo Workflow:

Step	Command	Purpose
1	<code>smart-tests record build</code>	Groups related test sessions
2	<code>smart-tests record session</code>	Creates test session in CloudBees Uses <code>--observation</code> mode
3	<code>pytest --collect-only -qq</code>	Lists all available tests

Step	Command	Purpose
4	<code>smart-tests subset pytest</code>	Generates predicted test subset
5	<code>pytest @subset.txt</code>	Runs tests (all in observation mode)
6	<code>smart-tests record tests</code>	Uploads results to CloudBees

Demo App Implementation:

```
# .github/workflows/tests.yml (simplified)

steps:
  - name: Record build
    run: smart-tests record build --build ${ github.run_id }

  - name: Record session
    run: |
      smart-tests record session \
        --build ${ github.run_id } \
        --observation \
        --test-suite pytest-unit > session.txt

  - name: Collect tests
```

Warning

COMMON MISCONCEPTIONS: Technical Integration

Misconception #1: "I need to modify my pytest tests to work with Smart Tests."

Reality: Zero test code changes required. Smart Tests uses standard pytest commands (`--collect-only` , `--junit-xml`) that already exist.

Misconception #2: "The six-step workflow only works on GitHub Actions."

Reality: The workflow is CI-agnostic. The Smart Tests CLI commands are identical on Jenkins, GitLab CI, CircleCI, Azure Pipelines, etc. Only the CI platform syntax changes.

Misconception #3: "I need to change my test organization to use Smart Tests."

Reality: Smart Tests adapts to YOUR test structure. Keep your directories, markers, and pytest.ini configuration as-is.

Misconception #4: "Smart Tests requires installing agent software on CI runners."

Reality: Smart Tests CLI is a simple Python package (`pip install smart-tests-cli`). No agent, daemon, or infrastructure changes needed.

Misconception #5: "JUnit XML slows down my tests."

Reality: JUnit XML generation (`--junit-xml`) is built into pytest and adds negligible overhead (<1% of runtime).

pytest-Specific Integration Points

Why These Patterns Work:

Smart Tests leverages standard pytest features, requiring no test code changes.

Test Collection

Concept: pytest can list all tests without running them

```
# Standard pytest feature
pytest --collect-only -qq [path/to/tests]

# Output format (one test per line)
tests/unit/test_auth.py::test_login
tests/unit/test_auth.py::TestAuthService::test_validate_token
tests/integration/test_api.py::test_create_user
```

Test Execution from File

Concept: pytest can read test list from a file

```
# Create test list file
echo "tests/unit/test_auth.py::test_login" > subset.txt
echo "tests/unit/test_user.py::test_create" >> subset.txt

# Run tests from file
pytest @subset.txt
```

JUnitXML Output

Concept: Smart Tests requires JUnit XML for result parsing

```
# Required flag
pytest --junit-xml=results.xml

# Smart Tests compatibility setting
pytest -o junit_family=legacy --junit-xml=results.xml
```

Why **junit_family=legacy** :

- Smart Tests parses JUnit XML format
- Legacy format has most compatibility
- Newer formats (xunit2) may work but legacy is guaranteed

CLI Installation and Configuration

Installation:

```
# Install Smart Tests CLI (Python 3.7+)
pip install smart-tests-cli~2.0

# Or in your requirements.txt
# smart-tests-cli~2.0

# Verify installation
smart-tests --version
smart-tests verify
```

Environment Configuration:


```
# Set API token (choose one method)

# Method 1: Environment variable
export SMART_TESTS_TOKEN="<YOUR_SMART_TESTS_TOKEN>"

# Method 2: CI secret (GitHub Actions example)
# secrets.SMART_TESTS_TOKEN

# Method 3: CLI argument (not recommended for security)
# smart-tests record build --api-token "<YOUR_SMART_TESTS_TOKEN>"
```

Connectivity Test:

```
# Verify Smart Tests connection
smart-tests verify

# Expected output:
# Connected to CloudBees Smart Tests
# Organization: <YOUR_ORG_NAME>
# PTSv2: Enabled
```

Tip

KNOWLEDGE CHECK: Technical Integration Patterns

Test your technical integration knowledge:

- What is the correct order of the six-step Smart Tests workflow?**
 - ☐ Collect tests, Create subset, Record build, Record session, Run tests, Record results
 - ☒ Record build, Record session, Collect tests, Create subset, Run tests, Record results
 - ☐ Record session, Collect tests, Run tests, Create subset, Record build, Record results
 - ☐ Create subset, Collect tests, Run tests, Record results, Record build, Record session
- What command lists all pytest tests without running them?**
 - ☐ `pytest --list`
 - ☒ `pytest --collect-only -qq`
 - ☐ `pytest --dry-run`
 - ☐ `pytest --show-tests`
- What format does Smart Tests require for test results?**
 - ☐ TAP (Test Anything Protocol)
 - ☐ pytest's default output
 - ☒ JUnit XML
 - ☐ JSON format
- Which Smart Tests CLI command creates the predicted test subset?**
 - ☐ `smart-tests predict`
 - ☒ `smart-tests subset pytest`
 - ☐ `smart-tests select`
 - ☐ `smart-tests analyze`
- True or False: You need to modify your pytest test code to use Smart Tests.**
 - ☐ True
 - ☒ False - Smart Tests uses standard pytest features

Implementation Best Practices

Organizing pytest Tests for Smart Tests

Recommended Structure:

Pattern	Rationale
Separate test suites	Unit, integration, E2E as different sessions
Clear naming	Helps identify which tests are affected
Logical grouping	By feature, module, or layer
Consistent markers	Enable selective execution

Demo App Structure:

```
tests/
  unit/          -> pytest-unit session
    test_user_service.py
    test_project_service.py
    ...
  integration/   -> pytest-integration session
    test_auth_api.py
    test_projects_api.py
    ...
  e2e/           -> pytest-e2e session
    test_auth_e2e.py
    ...
```

Observation Mode Strategy

Phase 1: Initial Data Collection (Weeks 1-2)

Goal: Establish baseline patterns

Actions:

[] Enable observation mode in CI

[] Run on all branches (main, develop, feature)

[] Include both passing and failing builds

[] Collect at least 50 test sessions

Monitoring:

- Verify data appears in CloudBees dashboard

- Check Smart Tests CLI logs for errors

- Ensure JUnit XML upload succeeds

Phase 2: Accuracy Evaluation (Weeks 3-4)

Goal: Validate prediction quality

Analysis:

- [] Review accuracy metrics per change type
- [] Identify patterns of high/low accuracy
- [] Check if false negatives are acceptable
- [] Calculate potential time savings

Metrics to Track:

- Overall accuracy percentage
- Accuracy by test suite (unit vs integration)
- Accuracy by change size (small vs large commits)
- False negative rate

Phase 3: Production Readiness (Week 5+)

Goal: Make go/no-go decision

Evaluation Criteria:

- [] Accuracy > 90% consistently
- [] Time savings > 30%
- [] Team understands metrics
- [] Fallback process defined
- [] Stakeholder approval

Decision Matrix:

High accuracy + High savings = ENABLE

High accuracy + Low savings = ENABLE (low risk)

Low accuracy + High savings = WAIT (collect more data)

Low accuracy + Low savings = INVESTIGATE (may need tuning)

Production Deployment Approach

Gradual Rollout Strategy:

Phase	Scope	Success Criteria
Pilot	One non-critical project	No missed failures for 2 weeks
Expansion	10% of projects	Accuracy maintained, savings realized
Majority	50% of projects	Team confident, metrics stable
Full	All projects	Standard practice, automated monitoring

Demo App Timeline Example:

Week 1–2: Enable observation mode

Week 3–4: Review 100+ test sessions

Week 5: Accuracy validation (>95% observed)

Week 6: Enable predictions for patch-demo branch

Week 7: Enable for all feature branches

Week 8: Enable for main branch

Monitoring and Maintenance

Continuous Monitoring:

Metric	Action If Degraded
Accuracy trends	Investigate if drops below 85%

Metric	Action If Degraded
False negative rate	Alert if critical tests missed
Time savings realization	Verify actual vs predicted savings
Subset selection rate	Too high (>80%) = limited savings, Too low (<20%) = may be too aggressive

Regular Reviews:

Weekly:

- Check accuracy dashboard
- Review any false negatives
- Monitor time savings trends

Monthly:

- Calculate ROI realization
- Review with development teams
- Adjust target percentages if needed

Quarterly:

- Comprehensive effectiveness review
- Compare pre/post adoption metrics
- Plan optimizations

▼

Demo Patch Management

Branch Naming Rules:

- Must start with `patch-*`
- Prevents accidental main branch modifications
- Keeps main clean for repeatability

Patch Independence:

- All patches are independent
- Can be applied in any order
- Can be combined for complex scenarios
- Easy to revert with `action: revert`

11/2

KNOWLEDGE CHECK: Implementation Best Practices

Test your implementation knowledge:

- What is the recommended observation mode duration before production?
 - ☐ 1-2 days
 - ☒ 1-2 weeks minimum
 - ☐ 1 hour
 - ☐ 6 months
- What accuracy threshold indicates readiness for production predictions?
 - ☐ 50%
 - ☐ 75%
 - ☒ 85-95% consistently
 - ☐ 100% (never miss a failure)
- How should you organize pytest tests for Smart Tests?
 - ☐ All tests must be in one directory
 - ☐ Tests must be alphabetically organized
 - ☒ Logical groupings (by layer, feature, or markers) with separate sessions
 - ☐ Random organization is fine
- What should you do if prediction accuracy drops below 85%?
 - ☐ Immediately disable Smart Tests
 - ☐ Ignore it and continue
 - ☒ Investigate root cause (flaky tests, framework changes, etc.)
 - ☐ Delete the failing tests
- Which subset selection rate indicates a problem?
 - ☐ 30-50% (optimal savings)
 - ☒ >80% (too conservative, minimal savings)
 - ☐ 10-20% (aggressive but acceptable)
 - ☐ Varies by project

Calculating Your Business Value

ROI Framework

Three Value Streams:

- Time Savings:** Faster feedback loops
- Cost Reduction:** Lower CI infrastructure spend
- Productivity Gains:** Developers wait less

ROI Calculation Worksheet:

Current State Analysis

Your Test Suite Profile:

1. Test Execution Time

Full suite runtime: _____ minutes

2. Execution Frequency

Commits per day: _____

Developers: _____

Runs per day: _____ (commits x team size)

3. Annual Execution Time

Daily: _____ minutes x _____ runs = _____ minutes

Annual: _____ minutes x 250 days = _____ hours

Smart Tests Impact Projection

Assumptions (adjust based on observation mode data):

Prediction accuracy: 90% (typical)

Time savings: 50% (varies by change patterns)

Adoption rate: 80% of commits (some always run full suite)

Projected Savings:

1. Time Reduction

New runtime: _____ minutes (50% of current)

Time saved per run: _____ minutes

2. Annual Time Savings

Daily savings: _____ minutes x _____ runs x 90% = _____ min

Demo App Example (For Reference)

Demo App Profile:

- Tests: 140+

- Runtime: 3 minutes (with artificial delays)

- Runs/day: 100 (hypothetical)

- Team: 10 developers

Smart Tests Impact:

- Time savings: 50% (1.5 minutes per run)

- Daily savings: 150 minutes (2.5 hours)

- Annual savings: 625 hours

- Value at \$100/hr: \$62,500/year

ROI:

First year value: \$62,500

Key Variables That Drive ROI:

Variable	Impact on ROI
Test Suite Size	Larger suites = more savings potential
Execution Time	Longer suites = more impactful reductions
Commit Frequency	More commits = savings compound faster
Team Size	Larger teams = more concurrent runs = more savings
CI Costs	Higher compute costs = infrastructure savings matter more
Accuracy	Higher accuracy = less risk, faster adoption

Additional Benefits

Beyond Time Savings:

- **Faster Feedback:** Developers get results sooner
- **Reduced Infrastructure:** Less CI compute time
- **Increased Confidence:** Catch regressions early
- **Better Developer Experience:** Less waiting for builds
- **Cost Reduction:** Lower CI/CD infrastructure costs

Tip

KNOWLEDGE CHECK: Calculating Your Business Value

Test your ROI calculation understanding:

1. What are the three main value streams of Smart Tests?

- ☒ Time savings, cost reduction, productivity gains
- ☐ Bug detection, code quality, developer happiness
- ☐ Faster builds, fewer failures, better reports
- ☐ Infrastructure savings, licensing costs, training time

2. How do you calculate current test suite cost?

- ☐ Number of tests x complexity factor
- ☒ Full runtime x runs per day x days per year x hourly developer cost
- ☐ CI infrastructure cost only
- ☐ License cost + training cost

3. If your full suite takes 10 minutes and Smart Tests reduces it 50%, saving \$50k/year, what is the payback period (assuming \$10k implementation cost)?

- ☐ 6 months
- ☒ ~2-3 months
- ☐ 1 year
- ☐ Immediate

4. What is a key "beyond time savings" benefit of Smart Tests?

- ☐ Eliminates need for code reviews
- ☐ Automatically fixes bugs
- ☒ Faster feedback to developers (waiting less for CI results)
- ☐ Reduces need for QA team

5. True or False: Observation mode has zero risk because all tests still run.

- ☒ True
- ☐ False

Advanced Topics

Creating Custom Patches

Purpose: Extend demo with project-specific scenarios

Documentation:

- [patches/CREATING_NEW_PATCHES.md](#) - Detailed guide
- [patches/AI_PATCH_GENERATION_PROMPT.txt](#) - AI assistant prompt
- [patches/README.md](#) - Existing patch documentation

Process:

1. Identify domain to modify (users, projects, issues, etc.)
2. Design breaking change (validation, business logic, etc.)
3. Create patch file with git diff
4. Test patch applies/reverts cleanly
5. Document expected test failures
6. Add to workflow dropdown options

Summary

What You Learned

Core Concepts:

1. **Predictive Test Selection:** AI analyzes code changes to predict affected tests
2. **Observation Mode:** Risk-free validation by running all tests while creating predictions
3. **Accuracy Metrics:** Measure effectiveness before enabling predictions
4. **pytest Integration:** Standard pytest features, no test code changes required
5. **CI/CD Patterns:** Six-step workflow adaptable to any platform

pytest-Specific Patterns:

- Test collection with `--collect-only`
- Test execution from files with `@filename`
- JUnit XML output with `--junit-xml`
- Works with any pytest configuration
- Compatible with all pytest plugins

Business Value:

- 50-80% time savings typical
- Lower CI infrastructure costs
- Faster developer feedback loops
- Measured with observation mode first
- ROI calculable with real data

Critical Success Factors

For Any Customer Implementation:

1. **PTSv2 Enabled:** Contact CloudBees to activate
2. **JUnit XML Output:** Required for result parsing
3. **Test Collection:** Must work with pytest discovery
4. **Observation Mode First:** Never skip validation phase
5. **Metric-Driven Decisions:** Use actual data, not assumptions

6. **Gradual Rollout:** Start small, expand with confidence

Additional Resources

Demo Repository \\https://github.com/cloudbees-ps/issues-tracker-app: **CloudBees Documentation:**
<https://docs.cloudbees.com/docs/cloudbees-smart-tests/latest/>

← [SDLC Metrics: CloudBees Workflows](#)

[Smart Tests: Java and Maven](#) →